

Crypto Trading Bot Using TradingView Webhooks and Python

Student name: _____

Course/section: _____

Date: _____

1. Overview of Trading Strategy

This project implements a demo-only crypto trading bot. The strategy uses two common technical indicators: Exponential Moving Averages (EMA) and Relative Strength Index (RSI). A buy signal is produced when the fast EMA crosses above the slow EMA and RSI is below the configured upper limit. A sell signal is produced when the fast EMA crosses below the slow EMA while a long position is open and RSI is above the configured lower limit.

The purpose of using EMA crossover is to identify changes in trend direction. The RSI filter reduces low-quality entries by avoiding buys when the market is already extremely overbought and avoiding sells when the market is extremely oversold. For this assignment, the bot is connected only to Binance Spot Testnet, so every trade is a demo trade.

2. Pine Script Logic

The Pine Script file is `pine/tradingview\_ema\_rsi\_strategy.txt`. It defines a TradingView strategy named "EMA RSI Webhook Bot (Demo Testnet)".

Main inputs:

- Fast EMA length: default 9
- Slow EMA length: default 21
- RSI length: default 14
- Buy filter: RSI below 70
- Sell filter: RSI above 30
- Webhook secret, symbol, and order quantity

The script plots the fast EMA and slow EMA on the chart. It also plots BUY and SELL markers when signals are triggered. When a signal happens, the script sends a JSON alert message using TradingView's `alert()` function. Example buy payload:

```
```json
{
 "secret": "change-this-secret",
 "signal": "buy",
 "symbol": "BTCUSDT",
 "quantity": "0.001",
 "strategy": "EMA_RSI_Webhook_Bot",
 "price": "65000.00",
 "timeframe": "15",
 "bar_time": "1710000000000"
}
```

## ## 3. Webhook Workflow Diagram

The system flow is:

```
```text
TradingView Pine Script -> Buy/Sell Alert -> JSON Webhook Payload
-> Flask Server -> Validate Secret and Signal -> Binance Spot Testnet
```

-> Order Response -> Local Trade History

The diagram source is in `docs/system\_flow.mmd`, and an SVG version is in `docs/system\_flow.svg`.

4. Python Code Explanation

The Python backend is implemented in `bot.py` using Flask. The server exposes a POST endpoint:

```
```text
/webhook/tradingview
```
```

When a request arrives, the bot:

1. Checks that the request body is JSON.
2. Saves the incoming alert to `data/alerts.jsonl`.
3. Verifies the webhook secret if `WEBHOOK\_SECRET` is configured.
4. Validates that the signal is either `buy` or `sell`.
5. Validates the symbol and quantity.
6. Sends a market order to Binance Spot Testnet when `DRY\_RUN=false`.
7. Saves the trade result to `data/trades.csv`.
8. Writes server events and errors to `logs/bot.log`.

The bot starts in `DRY\_RUN=true` mode by default. In dry-run mode, it records fake fills locally without sending requests to Binance. This is useful for proving webhook flow before using testnet API keys. The code refuses non-testnet Binance URLs, which helps enforce the assignment rule that real trading is not allowed.

5. Screenshots

Add screenshots after running the project:

Screenshot 1: TradingView chart with BUY/SELL markers.

Insert image here.

Screenshot 2: Webhook server logs showing a received alert.

Insert image here.

Screenshot 3: Binance Spot Testnet order confirmation.

Insert image here.

Screenshot 4: Local trade history in `data/trades.csv`.

Insert image here.

6. Conclusion and Improvements

The completed bot demonstrates an end-to-end automated demo trading workflow. TradingView generates signals using EMA crossover and RSI logic, sends JSON webhooks to the Flask server, and the Python bot records the alert and places demo market orders through Binance Spot Testnet.

Possible improvements include stronger position tracking, risk management rules, stop-loss and take-profit orders, account balance checks before placing trades, better backtesting across multiple market conditions, and deployment on a cloud

server with HTTPS.